

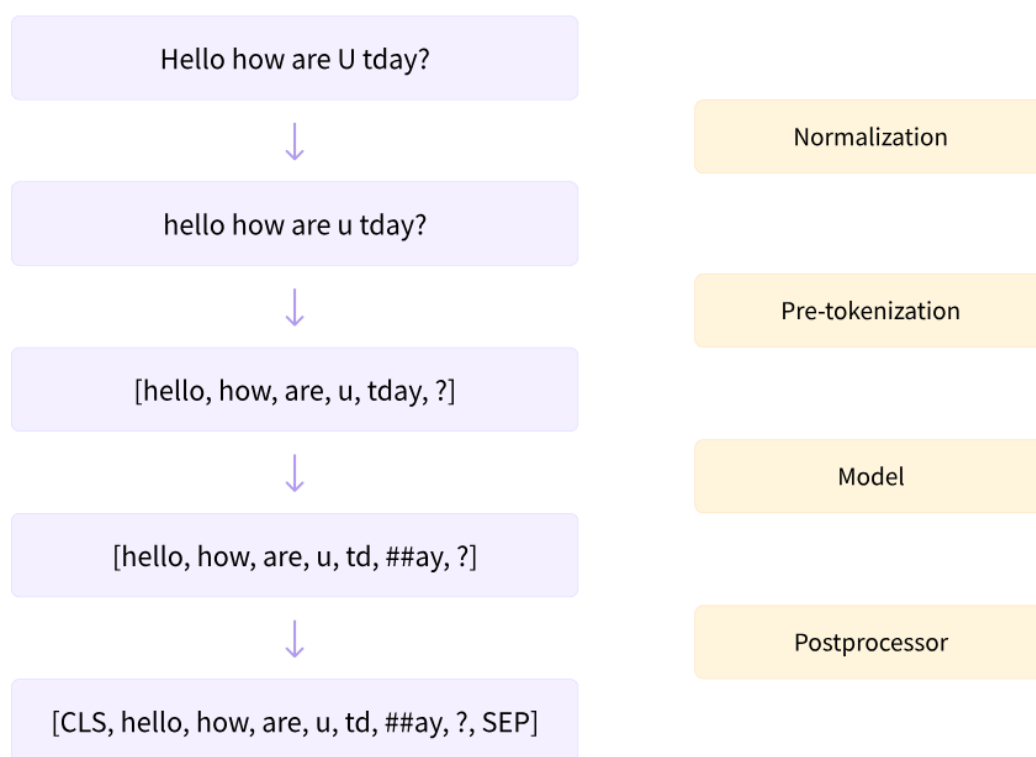
Normalization এবং Pre-tokenization

Transformer মডেলে ব্যবহৃত সবচেয়ে সাধারণ তিনটি subword tokenization algorithm হলো:

- Byte-Pair Encoding (BPE)
- WordPiece
- Unigram

এই algorithm গুলো বিস্তারিতভাবে বোঝার আগে, আগে জানা দরকার tokenizer টেক্সটকে প্রক্রিয়া করার সময় শুরুতে কী কী ধাপ অনুসরণ করে।

সাধারণভাবে tokenization pipeline-এর ধাপগুলো হলো:



অর্থাৎ, একটি টেক্সটকে মডেল অনুযায়ী ছোট ছোট subtoken-এ ভাগ করার আগে tokenizer সাধারণত দুইটি গুরুত্বপূর্ণ কাজ করে:

1. Normalization
2. Pre-tokenization

1. Normalization কী?

Normalization হলো টেক্সটকে একটি পরিষ্কৃত ও standard form-এ আনার প্রক্রিয়া।

এই ধাপে সাধারণত নিচের কাজগুলো করা হয়:

- অপ্রয়োজনীয় whitespace সরানো
- সব অক্ষর lowercase করা
- accent বা diacritic চিহ্ন সরানো
- Unicode normalization করা, যেমন NFC বা NFKC

সহজভাবে বললে, normalization-এর উদ্দেশ্য হলো এমনভাবে টেক্সটকে একরূপ করা যাতে পরে tokenizer সেটাকে সহজে এবং ধারাবাহিকভাবে process করতে পারে।

2. Transformers tokenizer-এর backend tokenizer

`Transformers` লাইব্রেরির tokenizer-এর মধ্যে `backend_tokenizer` নামে একটি attribute থাকে। এর মাধ্যমে `Tokenizers` লাইব্রেরির underlying tokenizer-এ access পাওয়া যায়।

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")  
print(type(tokenizer.backend_tokenizer))
```

আউটপুট:

```
<class 'tokenizers.Tokenizer'>
```

এর অর্থ হলো, `Transformers` tokenizer-এর ভেতরে আসলে `tokenizers.Tokenizer` কাজ করছে।

3. Normalization কীভাবে কাজ করছে তা দেখা

Tokenizer-এর `normalizer` attribute-এর মধ্যে `normalize_str()` নামে একটি method আছে। এটি ব্যবহার করে বোঝা যায়, একটি string-এর ওপর normalization প্রয়োগ করলে কী পরিবর্তন হয়।

```
print(tokenizer.backend_tokenizer.normalizer.normalize_str("Héllò hõw are ü?"))
```

আউটপুট:

```
'hello how are u?'
```

4. এখানে কী পরিবর্তন হলো?

আমরা যেহেতু `bert-base-uncased` tokenizer ব্যবহার করেছি, তাই এটি নিচের পরিবর্তনগুলো করেছে:

- সব অক্ষর lowercase করেছে
`Héllò` হয়েছে `hello`
- accent সরিয়েছে
`é, ò, ô, ü` বদলে গেছে `e, o, o, u`

তাই পুরো string:

`Héllò hôw are ü?`

পরিবর্তিত হয়ে হয়েছে:

`hello how are u?`

5. Pre-tokenization কী?

Normalization শেষ হওয়ার পর আসে pre-tokenization।

এই ধাপে টেক্সটকে প্রাথমিকভাবে এমনভাবে ভাঙা হয়, যাতে পরে tokenizer সহজে subtoken তৈরি করতে পারে।

Pre-tokenization সাধারণত নিচের কাজগুলো করতে পারে:

- space অনুযায়ী text আলাদা করা
- punctuation আলাদা করা
- word boundary চিহ্নিত করা

উদাহরণ হিসেবে:

`Hello, world!`

এটিকে pre-tokenization-এর মাধ্যমে এভাবে ভাগ করা হতে পারে:

- `Hello`
- `,`
- `world`
- `!`

অর্থাৎ, final subtokenization শুরু হওয়ার আগেই text-কে ছোট ছোট meaningful অংশে ভাগ করে নেওয়া হয়।

সহজ ভাষায় পুরো বিষয়টি

ধরো, tokenizer-এর কাজ হলো একটি বাক্যকে এমন ছোট ছোট অংশে ভাঙা, যাতে model সেটি বুঝতে পারে। কিন্তু সরাসরি ভাঙার আগে tokenizer আগে কিছু প্রস্তুতির কাজ করে।

প্রথম ধাপ: **Normalization**

এই ধাপে text পরিষ্কার করা হয়। যেমন:

- বড় হাতের অক্ষর ছোট করা
- accent সরানো
- অপ্রয়োজনীয় ফাঁকা জায়গা ঠিক করা

দ্বিতীয় ধাপ: **Pre-tokenization**

এই ধাপে text-কে প্রাথমিকভাবে ভাগ করা হয়। যেমন:

- শব্দ আলাদা করা
- punctuation আলাদা করা
- special character শনাক্ত করা

তৃতীয় ধাপ: **Subtokenization**

এখানে model-এর algorithm অনুযায়ী text-কে ছোট ছোট subtoken-এ ভাগ করা হয়।

Cased এবং Uncased tokenizer-এর পার্থক্য

এখানে একটি অনুশীলন দেওয়া হয়েছে:

`bert-base-cased` tokenizer load করে একই example চালিয়ে দেখতে বলা হয়েছে।

`bert-base-uncased`

এই tokenizer সাধারণত:

- uppercase ও lowercase-এর পার্থক্য রাখে না
- text-কে lowercase করে
- accent সরিয়ে দেয়

উদাহরণ:

`Héllò hòw are ü?`

হয়ে যায়

`hello how are u?`

`bert-base-cased`

এই tokenizer সাধারণত:

- uppercase ও lowercase অক্ষর ঠিক রাখে
- original casing ধরে রাখে
- অনেক সময় accent-ও রেখে দেয়

অর্থাৎ, **cased** version মূল text-এর আসল রূপ বেশি ধরে রাখে, আর **uncased** version text-কে সহজ এবং standard form-এ নিয়ে আসে।

প্রি-টোকেনাইজেশন

একটি tokenizer-কে শুধু raw text-এর উপর train করা যায় না। তার বদলে, প্রথমে আমাদের text-গুলোকে ছোট ছোট সম্ভায়, যেমন শব্দে, ভাগ করতে হয়। এখানেই pre-tokenization ধাপটি কাজে আসে। Chapter 2-এ যেমন আমরা দেখেছিলাম, একটি word-based tokenizer raw text-কে whitespace এবং punctuation-এর ভিত্তিতে সহজেই শব্দে ভাগ করতে পারে। tokenizer তার training-এর সময় যে subtokens শিখতে পারে, সেই শব্দগুলোই হবে তাদের boundary।

একটি fast tokenizer কীভাবে pre-tokenization করে তা দেখতে, আমরা tokenizer object-এর **pre_tokenizer** attribute-এর **pre_tokenize_str()** method ব্যবহার করতে পারি:

```
tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str("Hello, how are you?")
```

```
[('Hello', (0, 5)), (',', (5, 6)), ('how', (7, 10)), ('are', (11, 14)), ('you', (16, 19)), ('?', (19, 20))]
```

খেয়াল করুন, tokenizer ইতিমধ্যেই offset-গুলো track করছে, আর এভাবেই এটি আমাদের offset mapping দিতে পারে, যেটি আমরা আগের অংশে ব্যবহার করেছি। এখানে tokenizer দুটি space উপেক্ষা করে এবং সেগুলোকে মাত্র একটি দিয়ে প্রতিস্থাপন করে, কিন্তু **are** এবং **you**-এর মাঝে offset-এ jump দেখা যায়, যাতে সেটির হিসাব রাখা যায়।

যেহেতু আমরা একটি BERT tokenizer ব্যবহার করছি, তাই এখানে pre-tokenization-এর মধ্যে whitespace এবং punctuation-এর ভিত্তিতে split করা জড়িত। অন্য tokenizer-গুলোর ক্ষেত্রে এই ধাপের নিয়ম আলাদা হতে পারে। উদাহরণস্বরূপ, যদি আমরা GPT-2 tokenizer ব্যবহার করি:

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str("Hello, how are you?")
```

এটিও whitespace এবং punctuation-এর উপর split করবে, কিন্তু এটি space-গুলো ধরে রাখবে এবং সেগুলোকে **G** symbol দিয়ে প্রতিস্থাপন করবে, যাতে token decode করলে original space-গুলো আবার ফিরে পাওয়া যায়:

```
[('Hello', (0, 5)), (',', (5, 6)), ('Ghow', (6, 10)), ('Gare', (10, 14)), ('G', (14, 15)), ('Gyou', (15, 19)), ('?', (19, 20))]
```

এছাড়াও লক্ষ্য করুন, BERT tokenizer-এর মতো নয়, এই tokenizer double space উপেক্ষা করে না।

আরেকটি শেষ উদাহরণ হিসেবে, চলুন T5 tokenizer-টির দিকে তাকাই, যা SentencePiece algorithm-এর উপর ভিত্তি করে তৈরি:

```
tokenizer = AutoTokenizer.from_pretrained("t5-small")
tokenizer.backend_tokenizer.pre_tokenizer.pre_tokenize_str("Hello, how are you?")
```

```
[('_Hello,', (0, 6)), ('_how', (7, 10)), ('_are', (11, 14)), ('_you?', (16, 20))]
```

GPT-2 tokenizer-এর মতো, এটিও space ধরে রাখে এবং সেগুলোকে একটি নির্দিষ্ট token (**_**) দিয়ে প্রতিস্থাপন করে, কিন্তু T5 tokenizer শুধুমাত্র whitespace-এর উপর split করে, punctuation-এর উপর নয়। আরও লক্ষ্য করুন, এটি default হিসেবে বাক্যের শুরুতে (**He**llo-র আগে) একটি space যোগ করেছে এবং **are** ও **you**-এর মধ্যের double space উপেক্ষা করেছে।

এখন যেহেতু আমরা দেখলাম যে কিছু ভিন্ন tokenizer text কীভাবে process করে, আমরা এখন underlying algorithm-গুলো নিজে explore করা শুরু করতে পারি। আমরা শুরু করব বহুলভাবে প্রযোজ্য SentencePiece-এর একটি সংক্ষিপ্ত পরিচয় দিয়ে; তারপর, পরবর্তী তিনটি অংশে, subword tokenization-এ ব্যবহৃত তিনটি প্রধান algorithm কীভাবে কাজ করে তা পরীক্ষা করব।

SentencePiece

SentencePiece হলো text preprocessing-এর জন্য একটি tokenization algorithm, যেটি আপনি পরবর্তী তিনটি অংশে যে model-গুলো দেখব, সেগুলোর যেকোনোটির সঙ্গে ব্যবহার করতে পারেন। এটি text-কে Unicode character-এর একটি sequence হিসেবে বিবেচনা করে, এবং space-গুলোকে একটি বিশেষ character **_** দিয়ে প্রতিস্থাপন করে। যখন এটি Unigram algorithm-এর সঙ্গে একত্রে ব্যবহৃত হয় (section 7 দেখুন), তখন এটি এমনকি pre-tokenization ধাপও প্রয়োজন করে না, যা এমন ভাষার জন্য খুবই উপকারী যেখানে space character ব্যবহার করা হয় না (যেমন Chinese বা Japanese)।

SentencePiece-এর আরেকটি প্রধান বৈশিষ্ট্য হলো reversible tokenization: যেহেতু space-গুলোর জন্য কোনো বিশেষ আলাদা treatment নেই, তাই token decode করা হয় সেগুলোকে simply concatenate করে এবং **_**-গুলোকে space দিয়ে প্রতিস্থাপন করে — এর ফল হয় normalized text। আমরা আগেই দেখেছি, BERT tokenizer repeated space সরিয়ে দেয়, তাই তার tokenization reversible নয়।

Algorithm overview

পরবর্তী অংশগুলোতে, আমরা তিনটি প্রধান subword tokenization algorithm-এর মধ্যে গভীরে প্রবেশ করব: BPE (GPT-2 এবং অন্যদের দ্বারা ব্যবহৃত), WordPiece (উদাহরণস্বরূপ BERT-এ ব্যবহৃত), এবং Unigram (T5 এবং অন্যদের দ্বারা ব্যবহৃত)। শুরু করার আগে, নিচে দেওয়া হলো এগুলোর প্রত্যেকটি কীভাবে কাজ করে তার একটি সংক্ষিপ্ত overview। যদি এখনো বিষয়টি আপনার বোধগম্য না হয়, তাহলে পরের অংশগুলো পড়ার পর এই টেবিলে আবার ফিরে আসতে দ্বিধা করবেন না।

Model	BPE	WordPiece	Unigram
Training	একটি ছোট vocabulary দিয়ে শুরু করে এবং token merge করার rule শেখে	একটি ছোট vocabulary দিয়ে শুরু করে এবং token merge করার rule শেখে	একটি বড় vocabulary দিয়ে শুরু করে এবং token বাদ দেওয়ার rule শেখে
Training step	সবচেয়ে common pair-এর সঙ্গে সম্পর্কিত token-গুলো merge করে	pair-এর frequency-এর উপর ভিত্তি করে best score-যুক্ত pair-এর সঙ্গে সম্পর্কিত token-গুলো merge করে, এবং যেসব pair-এ প্রতিটি individual token কম frequent, সেগুলোকে অগ্রাধিকার দেয়	পুরো corpus-এর উপর গণনা করা loss-কে সর্বনিম্ন করেন এমন vocabulary-এর সব token সরিয়ে দেয়
Learns	Merge rules এবং একটি vocabulary	শুধু একটি vocabulary	প্রতিটি token-এর জন্য score-সহ একটি vocabulary
Encoding	একটি শব্দকে character-এ ভাঙে এবং training-এর সময় শেখা merge-গুলো প্রয়োগ করে	শুরু থেকে সবচেয়ে বড় subword খুঁজে যা vocabulary-তে আছে, তারপর শব্দের বাকি অংশের জন্য একই কাজ করে	training-এর সময় শেখা score ব্যবহার করে token-এ সবচেয়ে সম্ভাব্য split খুঁজে বের করে

এখন চলুন BPE-তে প্রবেশ করি।